fTechnological Institute of the Philippines
Quezon City

College of Computer Studies
Computer Science Department

CS CS 002 – Advanced Object-Oriented Programming
First Semester S.Y. 2026-2027

CONCEPT PROPOSAL

(1) Proponents

No.	Name	Role
1.	Erese, Daniel Noel B.	Project Manager / System Analyst UI/UX Designer and JavaFX Developer
2.	Lantin, Gabriel Jose	Lead Programmer (Core OOP and Business Logic) Quality Assurance Tester and Technical Writer
3.	Escobar, Miguel C.	Database Designer / DBA (MySQL Schema and DAO Layer)

(2) Title

EduReach: A Community Digital Literacy and Skill Assessment System

Desktop Application (JavaFX GUI) with MySQL Database | Supports UN SDG 4: Quality Education (Target 4.4)

(3) Background of the Study

Community learning centers, literacy non-profits, and vocational outreach programs provide non-formal education to out-of-school youth, adult learners, and job seekers who have limited access to formal digital skills training. In the Philippine setting, these include barangay learning hubs, community learning centers for alternative learning programs, and skills programs run by local government units (LGUs) and NGOs. Such centers are usually run by small teams and volunteer instructors, with tight budgets, shared computers, and unreliable internet access.

In the existing procedure, instructors record enrollment and attendance in paper logbooks, prepare tests on printed sheets or word-processor files, check answers by hand, encode scores in notebooks or loose spreadsheets, and issue informal certificates. This leads to the following problems encountered by the end users:

#	Problem in the existing procedure	Effect on end users
P1	**Fragmented, paper-based records and materials.** Enrollment, attendance, lessons, and test sheets are scattered across logbooks and personal files.	Records are lost, damaged, or duplicated. A learner's history is hard to retrieve, and materials disappear when volunteer instructors leave.
P2	**Unstandardized, hand-checked tests.**	Scoring is inconsistent and feedback is delayed. Instructors spend teaching hours on checking and encoding instead of instruction.
P3	**No consolidated progress tracking.**	Learners and instructors cannot see completed lessons, score trends, or which skills are mastered and which need review.
P4	**No verifiable credentials.**	Learners cannot show reliable proof of skills to employers, and employers cannot verify a certificate's authenticity.
P5	**Costly, cloud-dependent systems.** Enterprise learning management systems require licenses, servers, or constant broadband.	Small centers cannot afford or maintain them, so they stay with manual methods.
P6	**Weak access control and privacy of records.** Personal data sits in open logbooks and shared files.	Learners' information is exposed to unauthorized viewing, which conflicts with the Data Privacy Act of 2012 (RA 10173).
P7	**Materials and tools not designed for accessibility.** Printed sheets and timed paper tests offer no accommodation.	Learners with visual, motor, or other disabilities are effectively excluded from assessments.

(4) Project Objectives

General objective: To develop EduReach, an object-oriented desktop application with a graphical user interface and a relational database, that helps community learning centers deliver structured digital skills courses, assess learners automatically, and issue verifiable records of competency, in support of UN SDG 4 (Target 4.4: increasing the number of youth and adults with relevant skills for employment and decent jobs).

Specific objectives: Each objective answers one or more of the problems above and carries a measurable target.

Fixes	Objective	Proposed solution	Measurable target
P1	Centralize the learning repository and learner records	Store users, courses, lessons, enrollments, and attendance in a MySQL database. Courses and question banks can be cloned and reused by the next instructor.	Any learner record retrievable within 3 clicks; no paper logbook needed.

Fixes	Objective	Proposed solution	Measurable target
P2	Automate the assessment engine	Timed multiple-choice quizzes with randomized order, automatic scoring, and instant feedback on each answer, driven by the Gradable interface.	Result and feedback shown within 2 seconds of submission; 100% of multiple-choice items auto-scored.
P3	Track learner competency	Progress Tracker computes completion percentage, average scores, and skill-domain mastery, and awards skill badges.	Per-learner dashboard on login; class gradebook generated in under 10 seconds.
P4	Issue verifiable credentials	Generate PDF certificates and skill transcripts, each with a unique verification code that staff can check in the system.	Every issued certificate carries a unique, checkable code.
P5	Provide a low-cost, offline-first solution	A lightweight desktop app on a local MySQL database using free, open-source tools.	All core features work without internet; zero licensing cost.
P6	Enforce role-based security and data protection	Separate Student, Instructor, and Administrator privileges; salted password hashing; parameterized queries; audit logs; privacy notice at registration.	Zero SQL built by string concatenation; all logins and privileged actions logged.
P7	Make the system accessible to all users, including PWDs	Keyboard-only navigation, screen-reader labels, high-contrast theme, adjustable font size, extended-time accommodations for timed quizzes, and English/Filipino labels.	All screens fully operable by keyboard alone; accessibility checklist passed before release.

(5) Expected Output

The project will produce a working, reliable, efficient, user-friendly, and accessible desktop system, together with its supporting technical documentation.

5.1 Products and Services: Functional Modules

Module	Key functions	Users	OOP mapping
Authentication and Profile	Secure login with role-based menus, profile and password management, temporary lockout after repeated failed attempts.	All roles	User (abstract), UserFactory
Course Manager	Create, update, and categorize courses (e.g., Literacy, IT, Financial Basics); organize lessons and learning media; enroll learners; record session attendance.	Instructor, Student	CourseDAO, EnrollmentDAO, AttendanceDAO
Quiz Engine	Question banks, timed multiple-choice assessments, randomized order, automatic scoring, instant answer feedback, and skill-domain tagging.	Student, Instructor	Gradable interface; QuizModule, PracticalTask
Progress Tracker	Completion percentage, average and historical scores, skill-domain mastery, and badge awards.	Student, Instructor	ProgressReport; Observer for badge awarding

Module	Key functions	Users	OOP mapping
Credentials	PDF certificates and skill transcripts with unique verification codes.	Student, Instructor, Admin	CertificateGenerator
Reporting and Administration	Class gradebooks, center-wide metrics, CSV/PDF export, audit logs, user management, and database backup and restore.	Instructor, Administrator	AdminController, ReportGenerator, AuditLogDAO

Documentation outputs: UML use case and class diagrams, entity-relationship diagram, MySQL DDL and sample data scripts, source code repository, user manual, test report, and final project report.

5.2 Object-Oriented Design

The class hierarchy has an abstract User class (userID, fullName, email) extended by StudentUser (submitQuiz, viewProgress), InstructorUser (createCourse, gradeQuiz), and AdminUser (auditLogs, backupDB). The Gradable interface (calculateScore, getMaxScore) is implemented by QuizModule and PracticalTask.

OOP pillar	Application in EduReach
Encapsulation	Sensitive attributes such as passwordHash and scoreObtained are private and accessed only through getters and setters with strict input validation. Scores are computed inside the grading classes, so the UI cannot alter them directly.
Inheritance	The abstract User class holds shared attributes and behavior. StudentUser, InstructorUser, and AdminUser extend it and add only role-specific operations, reducing duplicated code.
Polymorphism	Gradable declares calculateScore(). QuizModule (auto-scored items) and PracticalTask (instructor rubric) override it differently, so the Progress Tracker handles every assessment type uniformly.
Abstraction	The Data Access Object (DAO) pattern hides all SQL and JDBC details behind interfaces such as CourseDAO, keeping MySQL queries out of the JavaFX controllers and allowing storage changes without touching the UI.

Supporting practices: MVC separates FXML views, controllers, and models; a Singleton DatabaseManager provides pooled connections; a Factory creates the correct user type at login; custom exceptions (e.g., InvalidCredentialsException, DuplicateEnrollmentException) handle errors gracefully; collections, generics, and streams support reporting; and JUnit 5 tests cover grading and DAO logic.

5.3 Database Design (MySQL 8.0)

The relational schema is normalized to Third Normal Form with full foreign key constraints and indexes on lookup fields (username, verification_code, and foreign keys). Passwords are stored as salted BCrypt hashes, and every query uses PreparedStatement to prevent SQL injection.

Table	Purpose	Key fields and relationships
users	Accounts and roles	user_id (PK), username (UNIQUE), password_hash, full_name, email, role (ENUM: STUDENT, INSTRUCTOR, ADMIN), is_active
skill_domains	Skill areas assessed	domain_id (PK), name (e.g., Basic Computer Use, Internet Safety, Financial Basics)
courses	Course units	course_id (PK), title, category, instructor_id (FK to users)
lessons	Content within a course	lesson_id (PK), course_id (FK), title, content_path, sequence_no
enrollments	Student-course link and status	enrollment_id (PK), student_id (FK), course_id (FK), status, enrolled_on; UNIQUE (student_id, course_id)
attendance	Session attendance	attendance_id (PK), enrollment_id (FK), session_date, status
quizzes	Timed assessments	quiz_id (PK), course_id (FK), title, time_limit_min, passing_pct
questions, question_options	Question bank and answer choices	question_id (PK), quiz_id (FK), domain_id (FK), question_text; option_id (PK), question_id (FK), option_text, is_correct
submissions, submission_answers	Quiz attempts and per-item answers	sub_id (PK), student_id (FK), quiz_id (FK), score_pct DECIMAL(5,2), submitted_at; per-answer rows link sub_id, question_id, selected_option_id
student_badges, certificates	Earned badges and issued credentials	student_id (FK), domain_id or course_id (FK), earned_on / issued_on, verification_code (UNIQUE)
audit_logs	Security and activity trail	log_id (PK), user_id (FK), action, logged_at

5.4 Quality Attributes

- **Reliable:** quiz submission and scoring run inside a database transaction so results are saved completely or not at all; strict input validation; custom exception handling; database backup and restore; automated unit tests.
- **Efficient:** pooled database connections, indexed queries, offline-first design, and a small footprint that runs on standard center computers.
- **User-friendly:** role-specific dashboards, consistent navigation, clear error messages and confirmations, and a short quick-start guide for volunteers.
- **Accessible to all users, especially PWDs:** full keyboard operation with visible focus, screen-reader labels through JavaFX accessibility properties, high-contrast and light/dark themes, adjustable font size, no information conveyed by color alone, per-student extended time for timed quizzes, and English/Filipino interface labels.
- **Secure:** role-based access control, salted password hashing, parameterized queries, audit logging, and a privacy notice aligned with RA 10173.

5.5 Technology Stack and Schedule

Component	Selected technology
Language	Java 17 or higher (single stack chosen for the whole team)
GUI toolkit	JavaFX with FXML views
Database and connector	MySQL 8.0 with JDBC and a connection pool (e.g., HikariCP)
Supporting tools	BCrypt for password hashing, a PDF library (e.g., OpenPDF) for certificates, JUnit 5 for testing, Maven or Gradle for builds, Git for version control

Phase	Weeks	Activities	Deliverables
1. Design and UML	W1-2	Finalize use cases, class diagrams, ERD, and MySQL DDL with sample data.	Approved UML set and DDL script
2. Core OOP and DAO	W3-4	Build abstract classes and interfaces, DatabaseManager with pooling, DAO CRUD logic, BCrypt authentication, and unit tests.	Working OOP core and data layer
3. GUI and Wiring	W5-6	Develop JavaFX views, connect controllers to DAOs, and build the quiz engine, progress tracker, and accessibility features.	Functional GUI prototype
4. Testing and Documentation	W7-8	Integration and exception-handling tests, SQL-injection and accessibility checks, PDF certificates, user manual, and final report.	Final system, manual, and report

(6) Significance of the Study

Many widely used learning platforms assume constant connectivity, server administration skills, or paid licenses, which small community centers rarely have. EduReach contributes a practical, low-cost model for non-formal education that works offline and is built on a documented object-oriented architecture. Its contributions include:

- **An offline-first delivery model** that lets centers with limited infrastructure use structured courses and automated assessment.
- **Competency-oriented assessment.** Quiz items are mapped to skill domains, so results describe what a learner can do (skill badges and transcripts), not only a numeric score.
- **Accessibility built in from the start** for learners with disabilities, addressing SDG Target 4.5 (equal access for vulnerable groups) alongside Target 4.4 (relevant skills for employment).
- **A reusable OOP reference design** (abstract User hierarchy, Gradable polymorphism, DAO abstraction, MVC) that future student developers can study and extend.

Possible uses of the results: learning centers can adopt and extend the system for their own courses; LGUs and NGOs can use its aggregate reports for program evaluation and funding reports; the schema and design can be adapted to other short vocational training programs; and

future researchers can extend it with adaptive testing, additional languages, or multi-computer deployment over a local network.

(7) Target Beneficiaries

Beneficiary	Who	Expected outcomes
Students / Learners	Out-of-school youth, adult learners, job seekers, and learners with disabilities	Structured self-paced modules, instant feedback on quizzes, visible progress and skill badges, and verifiable certificates to show employers.
Instructors / Volunteers	Volunteer and staff teachers at community centers	Less time on checking and record-keeping, reusable courses and question banks, automatic grading, ready-made gradebooks, and continuity when volunteers change.
Center Administrators	Center coordinators and system administrators	Secure central records, audit logs, aggregate reports for stakeholders, and simple database backup and restore.
Local Employers and Community	Employers and hiring partners in the locality	Ability to verify a candidate's credentials by code, and a more job-ready local workforce.
Partner Organizations	LGUs, NGOs, and schools that support the centers	Data for program evaluation and a low-cost tool that can be replicated in other communities.

Overall, the expected effect of EduReach is that community learning centers spend less time on administration and more on teaching, and that learners leave with recorded, verifiable proof of the digital skills they have gained.

(8) Remarks

To be filled out by the Faculty In-Charge.

Noted by:

Mr. Jess N. Garcia
Faculty In-Charge